

Log Analysis Report

ADMIN

Production • Last 24 hours

Report Date

07 May 2026, 08:11 UTC

Generated At

07 May 2026, 08:11 UTC

Role: ADMIN | Env: Production

Services (All Services — Admin Access)

Service	Errors	Warnings	Req Approved	Req Rejected	Users Added
log-controller-service	15	257	0	0	0
inventory-service	1	0	0	0	0
ide-service	1	0	0	0	0

System Health Overview

503	17	257	229	HEALTHY
Total Logs	Errors	Warnings	Info	Overall Status

Key Insights

■ Elevated Error Rate

Error rate is 3.4% (17/503 logs) — above the 1% caution threshold. Monitor closely.

■ ■ Warning Saturation

257 warnings vs 17 errors (15:1 ratio). High warning volume often precedes failures — inspect deprecation and saturation signals now.

■ Dominant Error Pattern

'RAW INPUT → service: [null], level: [null]' accounts for 41% of all errors (7 occurrences). Resolving this single error type would eliminate the majority of error volume.

■ Cross-Service Impact

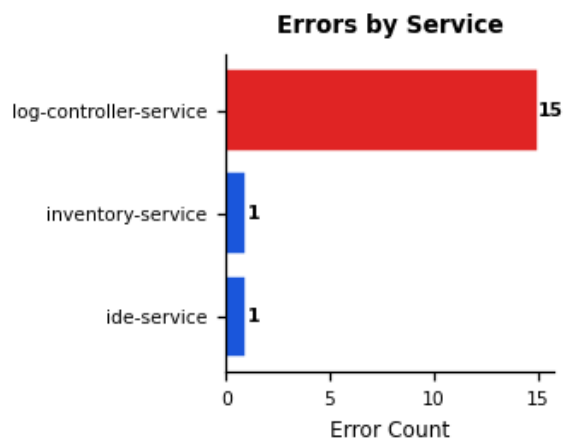
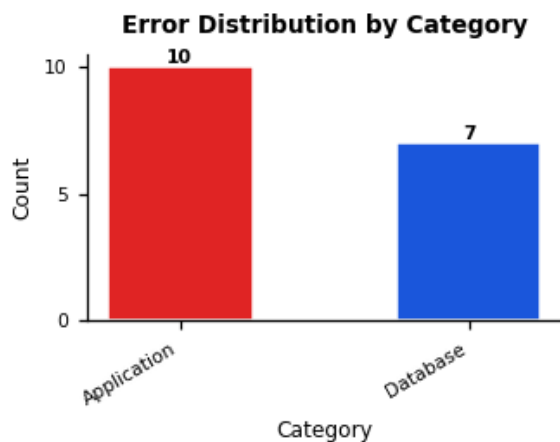
3 service(s) are currently reporting errors or warnings. Review the service dependency graph for potential cascading failures.

Top Errors (by Frequency)

#	Error Message	Service	Category	Count	% of Total
---	---------------	---------	----------	-------	------------

1	RAW INPUT → service: [null], level: [null]	log-controller-service	Application	7	41.2%
2	FINAL QUERY JSON → Query: {"bool":{"filter":...	log-controller-service	Database	4	23.5%
3	Aggregated 98 earlier error logs	log-controller-service	Application	1	5.9%
4	Aggregated 79 earlier error logs	inventory-service	Application	1	5.9%
5	Aggregated 2 earlier error logs	ide-service	Application	1	5.9%
6	FINAL QUERY JSON → Query: {"bool":{"filter":...	log-controller-service	Database	1	5.9%
7	FINAL QUERY JSON → Query: {"bool":{"filter":...	log-controller-service	Database	1	5.9%
8	FINAL QUERY JSON → Query: {"bool":{"filter":...	log-controller-service	Database	1	5.9%

■ Analysis Visualisations



Recommendations & Fixes

HIGH

1. RAW INPUT → service: [null], level: [null] [log-controller-service] (7 occurrences)

Why it happened: NullPointerException or missing expected value — an object was accessed before being initialised, or optional data was assumed to be present.

How to fix:

1. Locate the full stack trace in the logs to identify the exact class and line.
.
2. Add null checks or use Optional<> / Optional.ofNullable() before dereferencing the object.
.
3. Add input validation at the API / service boundary so null values are rejected early with a descriptive error.
.
4. Write a unit test reproducing the null condition to prevent regression.
.

MEDIUM

2. FINAL QUERY JSON → Query: {"bool":{"filter":{"range":{"@timestam [log-controller-service] (4 occurrences)

Why it happened: Application-level error — exact root cause requires log inspection to identify the originating module.

How to fix:

1. Search for the full stack trace of 'FINAL QUERY JSON → Query: {"bool":{"filter":{"range":{" in the log management system.
.
2. Identify the service, class, and method that originates this error.
.
3. Determine if the error is transient (add retry with exponential back-off) or deterministic (fix the root cause directly).
.
4. Add a targeted alert rule so future occurrences are flagged immediately.
.

HIGH

3. Aggregated 98 earlier error logs [\[log-controller-service\]](#) (1 occurrences)

Why it happened: The log-controller-service batched a large number of historical entries into a single event — typically caused by a log pipeline restart or catch-up flush after a service crash or rolling restart.

How to fix:

- 1 Cross-reference the timestamp with recent deployments, pod restarts, or crash events: `kubectl describe pod` .
.
- 2 Check log-controller-service buffer config (`max_buffer_size`, `flush_interval`) — large buffers produce bulk flushes on restart.
.
- 3 Review container restart history to confirm whether a crash triggered the flush.
.
- 4 If recurring, reduce the buffer window or migrate to a real-time log shipper (Fluentd / Logstash) to avoid batching.
.

MEDIUM

4. High Warning Volume (257 occurrences)

Why it happened: Warning count is disproportionately high relative to errors, which often signals impending failures or unresolved deprecations.

How to fix:

- 1 Filter warning logs by message pattern to identify the top warning types.
.
- 2 Investigate deprecation warnings — APIs or configs that will break in a future version.
.
- 3 Look for resource-saturation warnings (disk, memory, CPU) and provision capacity if needed.
.
- 4 Set up a warning-rate alert (e.g. > 50 per minute) to catch spikes before they escalate to errors.
.

LOW

5. Operational Recommendation

How to fix:

- 1 Inspect recent log-controller-service deploys, dependencies, and retries if its error count keeps rising.
.

LOW

6. Operational Recommendation

How to fix:

- 1 Correlate the top error message with infrastructure changes and upstream service failures.

LOW

7. Operational Recommendation

How to fix:

- 1 Warnings outnumber errors — inspect deprecation and saturation signals early.